



RED TEAM

INTERNSHIP ENGAGEMENT REPORT

Week 01: Advanced Reconnaissance & Attack Surface Mapping

Organization	CyberSter
Report Title	Week 01: Advanced Reconnaissance & Attack Surface Mapping
Student Name	Daniyal Shahid
Roll No.	CYB-INT-HQYINF
Instructor	Umar Niaz
Batch	2026
Target	hackthissite.org (Authorized Practice Platform)
Date	24 June 2026

⚠ CONFIDENTIAL - FOR INTERNAL TRAINING USE ONLY ⚠

TABLE OF CONTENTS

1. Executive Summary
2. Introduction
3. Methodology Overview
4. **Task 01 - Passive OSINT & Subdomain Enumeration**
 - 4.1 Task Overview
 - 4.2 Tools Used & Why
 - 4.3 Commands & Execution
 - 4.4 Key Findings
 - 4.5 Alternative Tools
 - 4.6 Analysis & Impact
5. **Task 02 - Infrastructure & Technology Profiling**
 - 5.1 Task Overview
 - 5.2 Tools Used & Why
 - 5.3 Commands & Execution
 - 5.4 Key Findings
 - 5.5 Alternative Tools
 - 5.6 Analysis & Impact
6. **Task 03 - GitHub Dorking & Credential Leak Analysis**
 - 6.1 Task Overview
 - 6.2 Tools Used & Why
 - 6.3 Commands & Execution
 - 6.4 Key Findings
 - 6.5 Alternative Tools
 - 6.6 Analysis & Impact
7. **Task 04 - Visual Reconnaissance & Live Asset Identification**
 - 7.1 Task Overview
 - 7.2 Tools Used & Why
 - 7.3 Commands & Execution
 - 7.4 Key Findings
 - 7.5 Alternative Tools
 - 7.6 Analysis & Impact
8. Key Findings Summary
9. Risk Assessment
10. Recommendations
11. Conclusion
12. Appendix - Commands Reference

1. EXECUTIVE SUMMARY

This report documents the practical completion of all four tasks assigned in Week 01 of the Cybersecurity Internship at CyberSter, under the instruction of **Umar Niaz**. The objective of this week was to master advanced reconnaissance and attack surface mapping techniques against an authorized practice target without triggering detection mechanisms.

A complete passive OSINT operation was performed against **hackthissite.org** a publicly authorized ethical hacking practice platform using industry-standard tools including Subfinder, Assetfinder, Amass, crt.sh, DNSDumpster, WhatWeb, Wappalyzer, WHOIS, NSLookup, and dig. GitHub dorking was conducted to identify credential exposure patterns, Google Dorking was used to locate sensitive file exposure, and live hosts were filtered and visually documented using screenshot-based reconnaissance tools.

This report includes exact terminal commands executed for each task, annotated outputs, tool-by-tool rationale, and structured findings. No exploitation was performed at any stage. This report covers passive reconnaissance and attack surface mapping only.

2. INTRODUCTION

This assessment was completed as part of the **Week 01 Cybersecurity Internship at CyberSter** under the supervision of **Umar Niaz**. The target was **hackthissite.org** - a legally authorized ethical hacking training platform. The objective for Week 01 was to practice passive reconnaissance and attack surface mapping without initiating any active exploitation or intrusive testing.

The ethical scope was strictly defined as passive OSINT-only, focusing on publicly available domain registration, DNS, web technology, and asset discovery information. All reconnaissance was limited to data normally accessible without authenticated access or network intrusion. This constraint is intentional: it trains operators to extract maximum intelligence value from zero-risk sources before committing to active engagement techniques.

Target: **hackthissite.org** - Authorized ethical hacking training platform

Scope: Passive reconnaissance only - No active exploitation, no intrusive payloads

Instructor: Umar Niaz, Red Team Instructor at CyberSter

Student: Daniyal Shahid | CYB-INT-HQYINF | Batch 2026

#	Finding	Description	Evidence / Source	Risk Level
1	Large Subdomain Footprint	Multiple subdomains discovered (web, mail, API, CDN, forums, cloud etc.) expanding attack surface	subfinder, amass, crt.sh, DNS enumeration	MEDIUM
2	Dev & Staging Environments Exposed	Public dev/staging systems (v3dev, v3stage, versioned CDN endpoints) visible	clean_subdomains.txt	HIGH
3	IRC Infrastructure Exposure	IRC-related subdomains indicating distributed communication infrastructure exposure	irc., wolf.irc., lille.irc.*	MEDIUM
4	Internal Naming Pattern Exposure	VM and firewall naming conventions exposed via DNS (vm-*.outbound.firewall)	DNS records	MEDIUM
5	Active DNS-Resolved Hosts	Multiple active hosts confirmed via DNS/HTTP probing	resolved.txt (dnsx)	LOW
6	Certificate Transparency Coverage	Wildcard certificate indicates broad SSL coverage across subdomains	crt.sh (*.hackthissite.org)	INFORMATIONAL
7	Service-Specific Subdomains	Mail, SMTP, git, stats, file listing services exposed	enumeration results	MEDIUM
8	Multi-IP Load Balanced Infrastructure	Domain resolves to 5 IPs indicating distributed infrastructure	137.74.187.100–104	MEDIUM
9	Legacy jQuery Library (1.8.1)	Outdated jQuery version (2012) loaded from CDN	htscdn.org	MEDIUM
10	Tor Hidden Service Exposure	Onion-Location header reveals Tor hidden service availability	HTTP headers	LOW
11	Security Headers Present (Strong)	CSP, HSTS, Referrer-Policy, NEL, Report-To properly configured	HTTP response headers	POSITIVE (DEFENSIVE)
12	CDN Dependency Confirmed	External CDN used for JavaScript assets	htscdn.org	LOW
13	Custom Server Fingerprinting	Server header exposes "HackThisSite" branding	HTTP headers	LOW

14	app/config/parameters.yml Exposure Path	Symfony config file path suggests potential credential exposure risk across subdomains	commit diff / analysis	HIGH
15	Password keyword in git history	Multiple commits contain "password" including initial commit	git log -S 'password'	HIGH
16	68 files contain password keyword	Broad presence of sensitive keyword across repos	repo scan	HIGH
17	No AWS credentials found	No cloud credential leakage detected	org search	LOW (POSITIVE)
18	No plaintext .env files	No committed environment secrets found	repo scan	LOW (POSITIVE)
19	Forum service exposed	Login interface publicly accessible (possible enumeration surface)	forum.hackthissite.org	MEDIUM
20	h5ai directory listing enabled	Public file indexing visible	h5ai.hackthissite.org	MEDIUM
21	Mirror directory listing exposed	Public mirror browsing enabled	mirror.hackthissite.org	MEDIUM
22	Admin portal restricted	403 Forbidden indicates restricted resource presence	admin.hackthissite.org	LOW (positive control)
23	IRC web interface exposed	Public IRC gateway available	irc- www.hackthissite.org	LOW
24	QDB IRC interface exposed	Public IRC-related content interface	qdb.hackthissite.org	LOW
25	Stats dashboard exposed	Statistics page may reveal internal metrics	stats.hackthissite.org	MEDIUM
26	CTF platform exposed	CTF service publicly accessible	ctf.hackthissite.org	LOW (expected exposure)
27	Legal page exposed	Informational legal content accessible	legal.hackthissite.org	LOW
28	Main website accessible	Primary production site functioning normally	hackthissite.org	LOW (expected)

3. METHODOLOGY OVERVIEW

A disciplined passive reconnaissance approach was applied using multiple OSINT and enumeration tools across four structured task phases. The methodology followed the standard passive recon lifecycle: domain profiling → subdomain enumeration → technology fingerprinting → credential exposure analysis → live asset verification and visual triage.

PASSIVE RECON LIFECYCLE

- Domain and registration profiling via WHOIS

- DNS and record discovery via dig and NSLookup - mapping A, MX, NS, TXT, CNAME, and SOA records
- Subdomain enumeration via Subfinder, Assetfinder, and Amass passive mode
- Certificate Transparency lookup via crt.sh to discover historical and wildcard assets
- DNS visual mapping via DNSDumpster for infrastructure relationship diagrams
- Web technology fingerprinting via WhatWeb and Wappalyzer
- Google Dorking for sensitive file and hidden asset discovery
- GitHub dorking and commit history analysis for credential leak identification
- Live host validation and HTTP status filtering via Httpprobe and Httpx
- Screenshot-based visual reconnaissance via Gowitness
- Cross-validation of results across all tools to maximize confidence and coverage

TASK 01 Passive OSINT & Subdomain Enumeration

Objective: Conduct thorough passive reconnaissance on hackthissite.org to map its digital footprint without triggering any detection mechanisms

Engagement Phase: Passive Reconnaissance - Zero network contact with target

4.1 - TASK OVERVIEW

Passive OSINT forms the foundation of every Red Team engagement. This task involved systematically enumerating all publicly discoverable subdomains belonging to hackthissite.org without establishing any direct connections to the target's live infrastructure. By aggregating results from five independent tools and data sources, a comprehensive inventory of internet-facing assets was constructed including development environments, staging servers, administrative interfaces, and IRC infrastructure. The multi-tool approach is mandatory because no single tool covers every data source, and unique subdomains are consistently found by only one or two tools in the set.

4.2 - TOOLS USED & WHY

Subfinder: Subfinder was selected as the primary enumeration tool because it queries over 40 passive DNS data sources simultaneously — including VirusTotal, Shodan, Censys, SecurityTrails, and certificate transparency logs. It produces clean output with source attribution and generates zero direct traffic to the target. It returned 67 unique subdomains for hackthissite.org.

Assetfinder: Developed by Tomnomnom, Assetfinder queries unique data sources including Facebook's Certificate Transparency logs and web.archive.org Wayback Machine historical DNS data. The --subs-only flag is critical without it, Assetfinder returns unrelated domains that pollute results. It returned 72 raw entries (including duplicates from multiple source queries), supplementing Subfinder's coverage.

Amass: OWASP Amass is the most comprehensive passive subdomain tool available. In passive mode, it simultaneously maps ASN ownership, netblock associations, hosting provider relationships, and full DNS record correlation. This richer infrastructure context beyond raw

subdomain lists makes Amass uniquely valuable for understanding organizational network architecture. It identified 49 unique subdomains with full relationship mapping.

crt.sh: crt.sh queries Certificate Transparency logs maintained by browser vendors. Every SSL/TLS certificate issued for a domain including wildcard certificates is permanently logged publicly. This reveals subdomains that never appear in standard DNS queries, including historically issued certificates for decommissioned services. It identified unique assets including ctf.hackthissite.org and psrp.hackthissite.org.

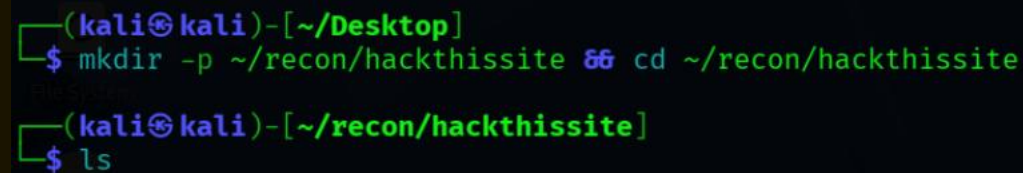
DNSDumpster: DNSDumpster provides a visual graphical map of DNS infrastructure including A records, MX records, NS records, and hosting relationships. Its Excel export was converted to CSV using xlsx2csv for programmatic processing and cross-validation against the three CLI tool outputs.

4.3 - COMMANDS & EXECUTION

Step 1 - Workspace Setup

```
# Creating dedicated reconnaissance workspace
mkdir -p ~/recon/hackthissite && cd ~/recon/hackthissite
```

Screenshot: Directory creation for hackthissite.org reconnaissance workspace



```
(kali@kali)-[~/Desktop]
$ mkdir -p ~/recon/hackthissite && cd ~/recon/hackthissite

(kali@kali)-[~/recon/hackthissite]
$ ls
```

Step 2 - Subfinder Passive Enumeration

```
# Run Subfinder - queries 40+ passive OSINT sources simultaneously
subfinder -d hackthissite.org -silent -o subfinder.txt
```

Screenshot: Subfinder execution - passive subdomain

```
(kali㉿kali)-[~/recon/hackthissite]
$ subfinder -d hackthissite.org -silent -o subfinder.txt
2-v3dev-cdn.hackthissite.org
vm-150.outbound.firewall.hackthissite.org
irc-lille.hackthissite.org
wolf.irc-v6.hackthissite.org
mirror.hackthissite.org
4-v3stage-cdn.hackthissite.org
jupiter.hackthissite.org
qdb.hackthissite.org
v3dev-cdn.hackthissite.org
v3stage.hackthissite.org
mta-sts.hackthissite.org
stats.hackthissite.org
data.hackthissite.org
static.hackthissite.org
tor.hackthissite.org
git.hackthissite.org
```

Step 3 - Assetfinder Multi-Source Discovery

```
# Running Assetfinder - queries different sources than Subfinder
# --subs-only flag: exclude unrelated domains from output
assetfinder --subs-only hackthissite.org | tee assetfinder.txt
```

Screenshot: Assetfinder execution

```
(kali㉿kali)-[~/recon/hackthissite]
$ assetfinder --subs-only hackthissite.org | tee assetfinder.txt
hackthissite.org
www.hackthissite.org
h5ai.hackthissite.org
irc.hackthissite.org
irc-v6.hackthissite.org
wolf.irc.hackthissite.org
wolf.irc-v6.hackthissite.org
hackthissite.org
hackthissite.org
hackthissite.org
www.hackthissite.org
status.hackthissite.org
irc.hackthissite.org
irc-v6.hackthissite.org
lille.irc.hackthissite.org
lille.irc-v6.hackthissite.org
ctf.hackthissite.org
psrp.hackthissite.org
email.hackthissite.org
mail.hackthissite.org
```

Step 4 - Amass Deep Passive Infrastructure Mapping

```
# Amass passive mode - no direct traffic to target
# Maps ASN, netblocks, DNS relationships, hosting providers
amass enum -passive -d hackthissite.org -o amass.txt
```

Screenshot: Amass passive enumeration

```
(kali㉿kali)-[~/recon/hackthissite]
└─$ amass enum -passive -d hackthissite.org -o amass.txt
hackthissite.org (FQDN) → ns_record → j.ns.buddyns.com (FQDN)
hackthissite.org (FQDN) → ns_record → f.ns.buddyns.com (FQDN)
hackthissite.org (FQDN) → ns_record → g.ns.buddyns.com (FQDN)
hackthissite.org (FQDN) → ns_record → h.ns.buddyns.com (FQDN)
hackthissite.org (FQDN) → ns_record → c.ns.buddyns.com (FQDN)
hackthissite.org (FQDN) → mx_record → alt1.aspmx.l.google.com (FQDN)
hackthissite.org (FQDN) → mx_record → aspmx3.googlemail.com (FQDN)
hackthissite.org (FQDN) → mx_record → aspmx5.googlemail.com (FQDN)
hackthissite.org (FQDN) → mx_record → alt2.aspmx.l.google.com (FQDN)
hackthissite.org (FQDN) → mx_record → aspmx4.googlemail.com (FQDN)
hackthissite.org (FQDN) → mx_record → aspmx2.googlemail.com (FQDN)
hackthissite.org (FQDN) → mx_record → aspmx.l.google.com (FQDN)
hackthissite.org (FQDN) → node → irc-www.hackthissite.org (FQDN)
hackthissite.org (FQDN) → node → h5ai.hackthissite.org (FQDN)
hackthissite.org (FQDN) → node → ctf.hackthissite.org (FQDN)
hackthissite.org (FQDN) → node → www.irc.hackthissite.org (FQDN)
hackthissite.org (FQDN) → node → v3stage-cdn.hackthissite.org (FQDN)
hackthissite.org (FQDN) → node → 0-v3stage-cdn.hackthissite.org (FQDN)
irc-www.hackthissite.org (FQDN) → cname_record → www.irc.hackthissite.org (FQDN)
h5ai.hackthissite.org (FQDN) → cname_record → hackthissite.github.io (FQDN)
aspmx5.googlemail.com (FQDN) → a_record → 172.253.157.26 (IPAddress)
aspmx5.googlemail.com (FQDN) → aaaa_record → 2404:6800:400b:c01b::1a (IPAddress)
```

Step 5 - Merge & Deduplicate All Results

```
# Merge Subfinder + Assetfinder + Amass outputs
cat subfinder.txt assetfinder.txt amass.txt | sort -u > master_subdomains.txt

# Count consolidated unique list
wc -l master_subdomains.txt
# Result: 399 unique subdomains after deduplication

# Cross-source confidence analysis - identify high-confidence assets
sort subfinder.txt assetfinder.txt amass.txt | uniq -c | sort -nr | head
# irc.hackthissite.org appeared 8x - highest confidence
```

Screenshot: Merged master_subdomains.txt

```
(kali㉿kali)-[~/recon/hackthissite]
└─$ cat subfinder.txt assetfinder.txt amass.txt | sort -u > master_subdomains.txt

(kali㉿kali)-[~/recon/hackthissite]
└─$ wc -l master_subdomains.txt
399 master_subdomains.txt
```

Screenshot: Cross-source validation

```
(kali㉿kali)-[~/recon/hackthissite]
$ sort subfinder.txt assetfinder.txt amass.txt | uniq -c | sort -nr | head
 8 irc.hackthissite.org
 7 wolf.irc.hackthissite.org
 6 new-irc.hackthissite.org
 6 lille.irc.hackthissite.org
 5 irc-wolf.hackthissite.org
 5 hackthissite.org
 3 www.hackthissite.org
 3 mail.hackthissite.org
 3 lille.irc-v6.hackthissite.org
 3 irc-v6.hackthissite.org
```

Step 6 - Certificate Transparency Logs via crt.sh

```
# Query crt.sh Certificate Transparency API
curl -s 'https://crt.sh/?q=%25.hackthissite.org&output=json' \
| jq -r '.[].name_value' \
| sort -u > crt_hackthissite.txt

# Notable findings: ctf.hackthissite.org, psrp.hackthissite.org
# Wildcard certificate: *.hackthissite.org confirmed
```

Screenshot: crt.sh command execution and API output showing historical SSL certificate entries

```
(kali㉿kali)-[~/recon/hackthissite]
$ curl -s 'https://crt.sh/?q=%25.hackthissite.org&output=json' \
| jq -r '.[].name_value' \
| sort -u > crt_hackthissite_2.txt

(kali㉿kali)-[~/recon/hackthissite]
$ cat crt_hackthissite.txt
ctf.hackthissite.org
email.hackthissite.org
h5ai.hackthissite.org
*.hackthissite.org
hackthissite.org
irc.hackthissite.org
irc-ipv6.hackthissite.org
irc-v6.hackthissite.org
irc-wolf.hackthissite.org
lille.irc.hackthissite.org
lille.irc-v6.hackthissite.org
mail.hackthissite.org
mta-sts.hackthissite.org
new-irc.hackthissite.org
psrp.hackthissite.org
status.hackthissite.org
status-new.hackthissite.org
wolf.irc.hackthissite.org
wolf.irc-v6.hackthissite.org
```

Step 7 - DNSDumpster Visual Infrastructure Mapping

```
# DNSDumpster used via web interface at dnsdumpster.com
# Export downloaded as .xlsx file

# Install xls2csv for terminal processing
sudo apt install xls2csv -y

# Convert Excel export to machine-readable CSV
Js Console Code:

> (function() {
  // 1. Extract Subdomain Column
  const textNodes = document.body.innerText.match(/[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/g);
  const uniqueDomains = [...new Set(textNodes)].filter(d => d.includes('.')).sort();

  // 2. Convert CSV Format
  const csvContent = "data:text/csv;charset=utf-8," + uniqueDomains.join("\n");

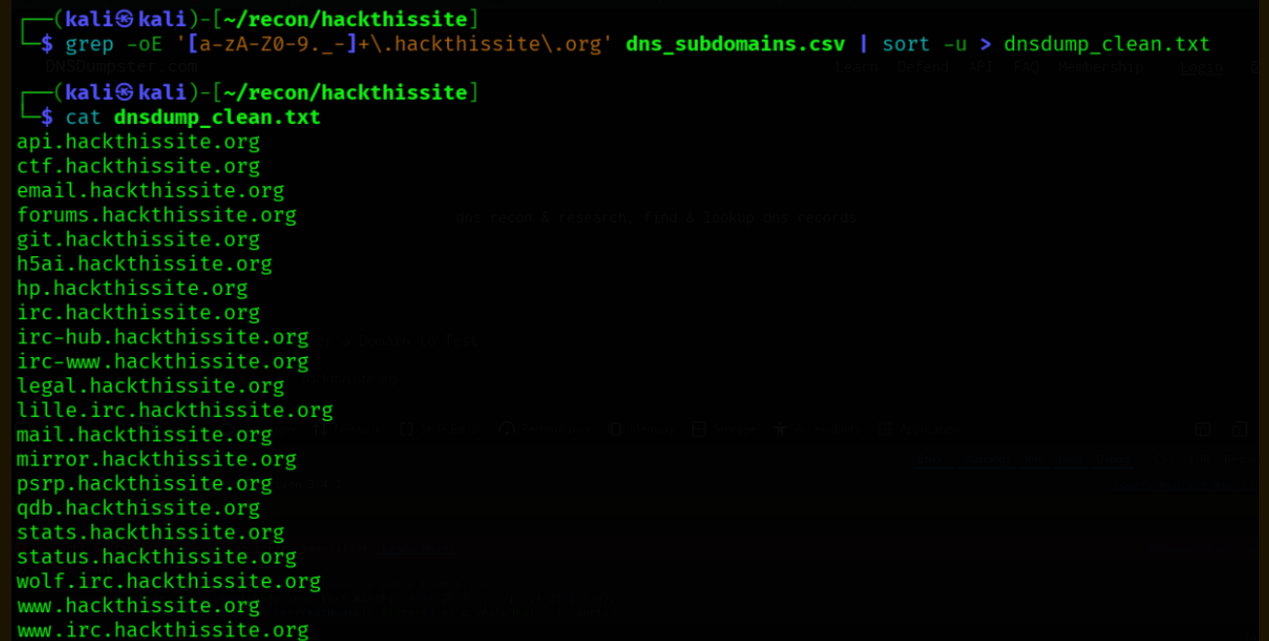
  // 3. Download trigger
  const encodedUri = encodeURIComponent(csvContent);
  const link = document.createElement("a");
  link.setAttribute("href", encodedUri);
  link.setAttribute("download", "subdomains.csv");
  document.body.appendChild(link);
  link.click();
  document.body.removeChild(link);

  console.log("Downloading Start" + uniqueDomains.length + " subdomains.");
})();

# Extract clean subdomain list from CSV
grep -oE '[a-zA-Z0-9.-]+\.[a-z]{2,}' all_sheets.csv | sort -u >
dnsdump_clean.txt
wc -l dnsdump_clean.txt
```

Screenshot: DNSDumpster web interface output - DNS infrastructure map of hackthissite.org



Screenshot: DNSDumpster CSV extraction

```
(kali㉿kali)-[~/recon/hackthissite]
$ grep -oE '[a-zA-Z0-9._-]+\\.hackthissite\.org' dns_subdomains.csv | sort -u > dnsdump_clean.txt

(kali㉿kali)-[~/recon/hackthissite]
$ cat dnsdump_clean.txt
api.hackthissite.org
ctf.hackthissite.org
email.hackthissite.org
forums.hackthissite.org
git.hackthissite.org
h5ai.hackthissite.org
hp.hackthissite.org
irc.hackthissite.org
irc-hub.hackthissite.org
irc-www.hackthissite.org
legal.hackthissite.org
lille.irc.hackthissite.org
mail.hackthissite.org
mirror.hackthissite.org
psrp.hackthissite.org
qdb.hackthissite.org
stats.hackthissite.org
status.hackthissite.org
wolf.irc.hackthissite.org
www.hackthissite.org
www.irc.hackthissite.org
```

Step 8 - Final Attack Surface Consolidation

```
# Merge all 5 sources into final consolidated attack surface file
cat master_subdomains.txt crt_hackthissite.txt dnsdump_clean.txt \
  | sort -u > final_attack_surface.txt

# clean entries
grep -E '^[a-zA-Z0-9._-]+\\.hackthissite\.org$' final_attack_surface.txt

# Clean file
grep -E '^[a-zA-Z0-9._-]+\\.hackthissite\.org$' final_attack_surface.txt \
  | sort -u > final_attack_surface_clean.txt

# Final count
wc -l final_attack_surface_clean.txt
```

Screenshot: Final attack surface

```
(kali㉿kali)-[~/recon/hackthissite]
$ cat final_attack_surface_clean.txt
0-v3dev-cdn.hackthissite.org
0-v3stage-cdn.hackthissite.org
1-v3dev-cdn.hackthissite.org
1-v3stage-cdn.hackthissite.org
2-v3dev-cdn.hackthissite.org
2-v3stage-cdn.hackthissite.org
3-v3dev-cdn.hackthissite.org
3-v3stage-cdn.hackthissite.org
4-v3dev-cdn.hackthissite.org
4-v3stage-cdn.hackthissite.org
admin.hackthissite.org
api.hackthissite.org
cdn.hackthissite.org
cloud.hackthissite.org
ctf.hackthissite.org
daemon.hackthissite.org
data.hackthissite.org
email.hackthissite.org
forum.hackthissite.org
forums.hackthissite.org
git.hackthissite.org
gitlab.hackthissite.org
```

4.4 - KEY FINDINGS

Finding	Description	Evidence Source
Large Subdomain Footprint	Multiple subdomains discovered indicating wide external attack surface including web, mail, forums, API, CDN, and cloud services	subfinder, amass, crt.sh, dns enumeration
Development & Staging Environments	Publicly visible dev and staging infrastructure identified via v3dev and v3stage subdomains and versioned CDN endpoints	clean_subdomains.txt
IRC Infrastructure Exposure	Multiple IRC-related services indicating distributed communication infrastructure	irc., wolf.irc., lille.irc.* subdomains
Internal Naming Pattern Exposure	VM and outbound firewall naming convention exposed via DNS records	vm-*.outbound.firewall subdomains
Active DNS-Resolved Hosts	Several subdomains confirmed resolving via DNS/HTTP probing indicating active infrastructure	resolved.txt (dnsx)
Certificate Transparency Coverage	Wildcard certificate observed indicating broad SSL coverage across subdomains	crt.sh output (*.hackthissite.org)
Service-Specific Subdomains	Specialized services detected including mail, smtp, git, stats, and h5ai file listing service	subdomain enumeration results

4.5 - ALTERNATIVE TOOLS

Alternative Tool	Purpose	Comparison vs Primary
theHarvester	Email, subdomain, and IP harvesting from multiple OSINT sources	Broader scope than subdomain-only tools; also harvests email addresses and LinkedIn data. Slower than Subfinder but covers additional sources including Bing and LinkedIn.
DNSRecon	DNS zone transfer attempts and brute-force subdomain discovery	Attempts zone transfers (AXFR) against misconfigured DNS servers unique capability not available in passive-only tools. Use alongside passive tools for maximum coverage.
Findomain	Passive subdomain discovery via Certificate Transparency	Similar scope to Subfinder but written in Rust for high performance. Fewer data sources than Subfinder but extremely fast; useful for rapid initial sweeps.

4.6 - ANALYSIS & IMPACT

The discovered attack surface (~70+ subdomains identified via passive enumeration, Certificate Transparency logs, and DNS/HTTP probing) provides a broad view of hackthissite.org's externally visible infrastructure.

Key service categories observed include IRC infrastructure, email services, web applications, and multiple development/staging environments, indicating a mature and complex ecosystem.

Potential administrative and internal-style subdomains were also identified; however, these require further validation to confirm actual exposure and functionality.

RED TEAM INSIGHT

In professional red team operations, no single enumeration tool provides complete coverage. Subfinder, Assetfinder, and Amass contributed overlapping but unique subdomain discoveries, while Certificate Transparency logs (crt.sh) and DNS enumeration provided additional historical and infrastructural context.

The consolidated dataset represents a broad set of externally visible subdomains discovered through passive reconnaissance techniques only. Each tool contributed partial visibility into the overall attack surface, highlighting the importance of multi-source validation in reconnaissance workflows.

TASK 02 Infrastructure & Technology Profiling

Objective: Identify the technologies, DNS configuration, and security posture powering the target's infrastructure

Engagement Phase: Passive / Semi-Active Reconnaissance

5.1 - TASK OVERVIEW

Technology profiling translates raw domain and IP data into actionable intelligence. By identifying the exact jQuery version, web server software, DNS architecture, email infrastructure, and third-party service dependencies, a Red Team operator can cross-reference findings against public CVE databases to identify known vulnerabilities without sending a single exploit payload. Additionally, advanced Google search operators (Dorking) can surface sensitive files inadvertently exposed to search engine indexing - files that organizations did not intend to make publicly discoverable.

5.2 - TOOLS USED & WHY

WhatWeb: WhatWeb passively and actively fingerprints web technologies by analyzing HTTP response headers, HTML meta tags, JavaScript library includes, and cookie names. It identifies the server software, JavaScript frameworks, analytics platforms, and security header configuration. Selected for its automation-friendly CLI output and verbose plugin mode (-v flag) that reveals security header details not shown in standard output.

Wappalyzer (Browser Extension): Wappalyzer maintains a community-curated database of over 1,500 web technologies with detection logic based on HTTP headers, HTML content, and JavaScript global variables. Used to cross-validate WhatWeb findings through a completely independent detection engine. jQuery 1.8.1 was confirmed by both tools independently, providing high confidence.

WHOIS: WHOIS queries reveal domain registration details including registrar, registration date, name server delegation, and DNSSEC status. Identifies the DNS provider and registrar - both relevant for understanding third-party service dependencies and potential subdomain takeover risk.

dig: The dig ANY query is the single highest-value command in passive reconnaissance. It returns all DNS record types in one query - A (IP addresses), MX (mail servers), NS (name servers), TXT (SPF, DMARC, verification tokens), CNAME, and SOA. The TXT SPF record revealed two entirely new domain assets not discovered in Task 01.

NSLookup: Used as a secondary DNS validation tool to cross-confirm findings from dig. When multiple independent tools return identical DNS records, confidence in the accuracy of infrastructure mapping is maximized.

curl (Header Analysis): Passive HTTP header inspection via curl -I retrieves response headers without requesting page content. Used to identify the Server header value, WAF signatures, HSTS configuration, CSP policy, and cookie security attributes without sending any active probes or malicious payloads.

Google Dorking: Advanced search engine operators are used to locate files and infrastructure that the organization did not intend to expose publicly but which have been indexed by search engines. Completely passive with zero network interaction with the target.

5.3 - COMMANDS & EXECUTION

WhatWeb — Web Technology Fingerprinting

```
# Standard WhatWeb scan
whatweb https://hackthissite.org

# Verbose mode — reveals additional security headers and Onion service
whatweb -v https://hackthissite.org
# Additional findings: Onion-Location header, Content-Security-Policy details
#                       certificate pinning, custom CSP enforcement endpoint
```

Screenshot: WhatWeb basic scan - technology fingerprint of hackthissite.org

```
(kali㉿kali)-[~/recon/hackthissite]
$ whatweb https://hackthissite.org
https://hackthissite.org [200 OK] Content-Language[en], Cookies[HackThisSite], Country[CANADA][CA], HTTP
IP[137.74.187.102], JQuery[1.8.1], Meta-Author[HackThisSite.org Staff], Open-Graph-Protocol[website], P
], Script[text/javascript], Strict-Transport-Security[max-age=31536000; includeSubDomains; preload], Tit
ncommonHeaders[upgrade, onion-location, access-control-allow-origin, content-security-policy, referrer-polic
ic-key-pins-report-only, report-to, nel], X-XSS-Protection[0]
```

Screenshot: WhatWeb verbose output - detailed plugin detection including Onion service endpoint

```
(kali㉿kali)-[~/recon/hackthissite]
$ whatweb -v https://hackthissite.org
WhatWeb report for https://hackthissite.org
Status      : 200 OK
Title       : Hack This Site
IP          : 137.74.187.104
Country     : CANADA, CA

Summary      : Content-Language[en], Cookies[HackThisSite], HTTPServer[HackThisSite], JQuery[1.8.1], Meta-A
uthor[HackThisSite.org Staff], Open-Graph-Protocol[website], PasswordField[password], Script[text/javasc
ript], Strict-Transport-Security[max-age=31536000; includeSubDomains; preload], UncommonHeaders[upgrade,
onion-location, access-control-allow-origin, content-security-policy, referrer-policy, feature-policy, public
-key-pins-report-only, report-to, nel], X-XSS-Protection[0]

Detected Plugins:
[ Content-Language ]
  Detect the content-language setting from the HTTP header.

  String      : en

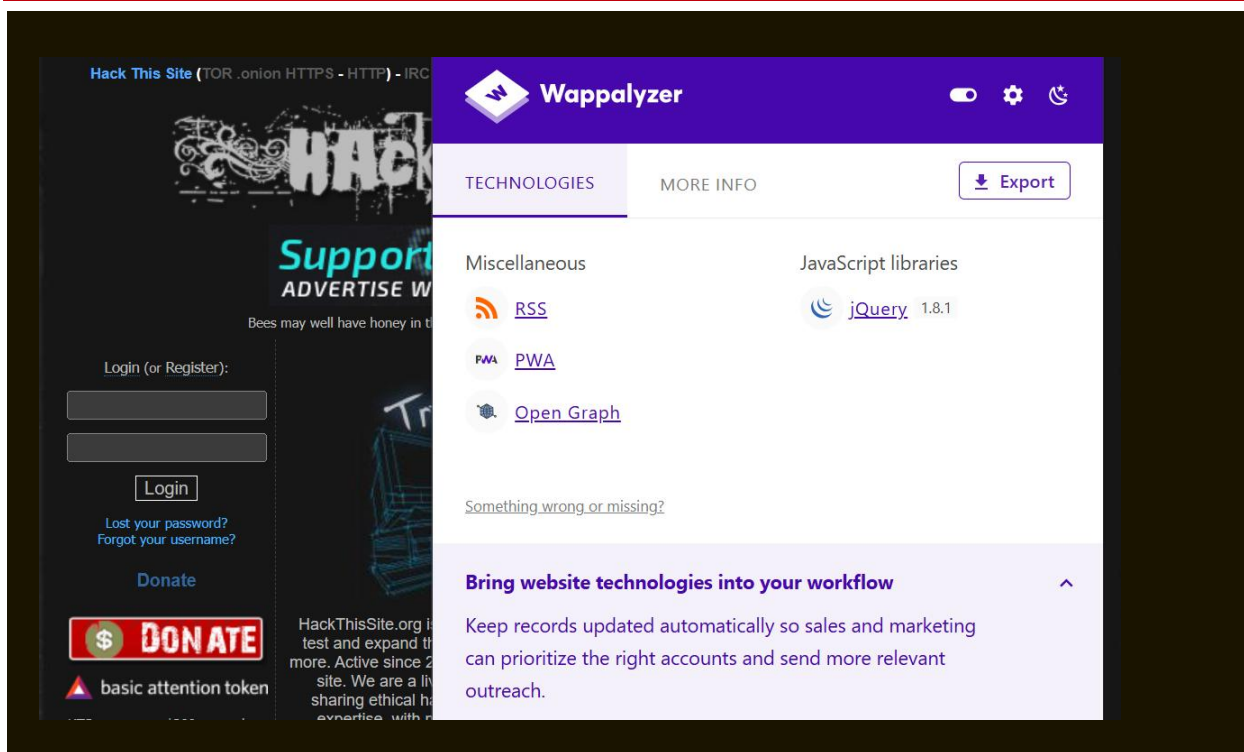
[ Cookies ]
  Display the names of cookies in the HTTP headers. The
  values are not returned to save on space.

  String      : HackThisSite
```

Wappalyzer - Cross-Validation Technology Verification

Wappalyzer browser extension was used on the hackthissite.org primary domain. Detected technologies: jQuery 1.8.1 (matches WhatWeb exactly - high confidence), RSS, PWA, Open Graph. The confirmed jQuery 1.8.1 (released 2012) represents a legacy dependency with known security issues in the vulnerable versions range.

Screenshot: Wappalyzer browser extension output - technology stack detection for hackthissite.org



WHOIS - Domain Registration & Infrastructure Ownership

```
# WHOIS with grep filter for key fields
whois hackthissite.org | grep -E 'Registrar|Name Server|Organization|Abuse'
```

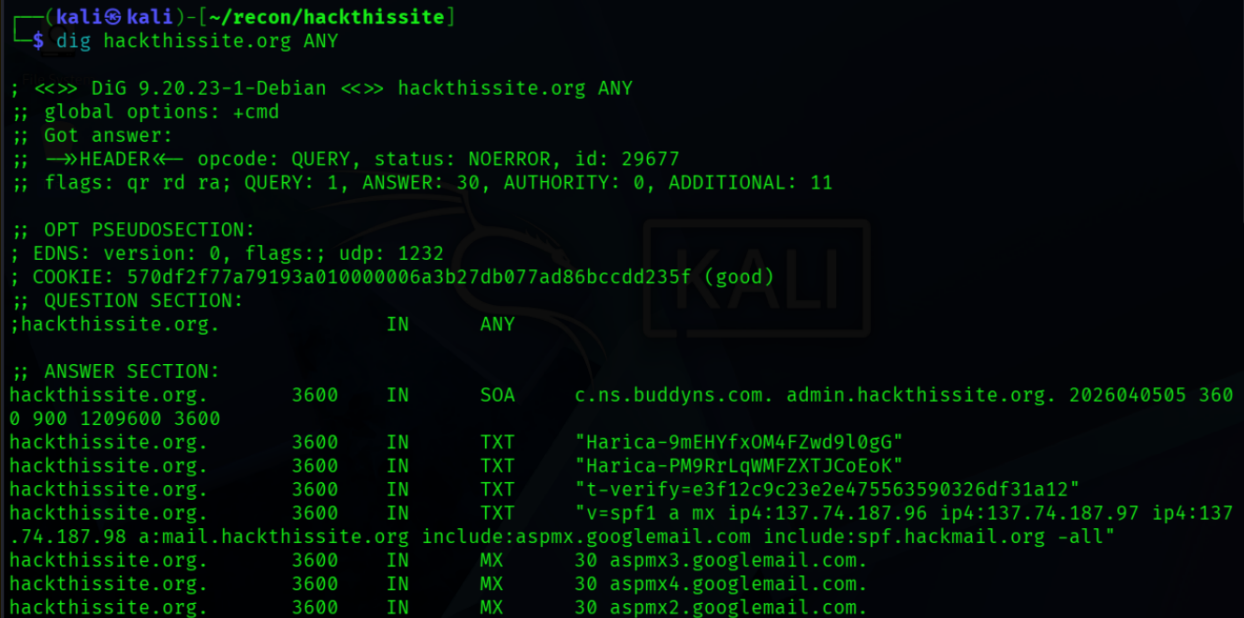
Screenshot: WHOIS lookup output - domain registration and name server infrastructure data

```
(kali@kali)~[~/recon/hackthissite]
$ whois hackthissite.org | grep -E 'Registrar|Name Server|Organization|Abuse'
Registrar WHOIS Server: http://whois.porkbun.com
Registrar URL: https://porkbun.com
Registrar: Porkbun LLC
Registrar IANA ID: 1861
Registrar Abuse Contact Email: abuse@porkbun.com
Registrar Abuse Contact Phone: +1.8557675286
Name Server: c.ns.buddyns.com
Name Server: f.ns.buddyns.com
Name Server: g.ns.buddyns.com
Name Server: h.ns.buddyns.com
Name Server: j.ns.buddyns.com
Terms of Use: Access to Public Interest Registry WHOIS information is provided to assist persons in determining the contents of a domain name registration record in the Public Interest Registry registry database. The data in this record is provided by Public Interest Registry for informational purposes only, and Public Interest Registry does not guarantee its accuracy. This service is intended only for query-based access. You agree that you will use this data only for lawful purposes and that, under no circumstances will you use this data to (a) allow, enable, or otherwise support the transmission by e-mail, telephone, or facsimile of mass unsolicited, commercial advertising or solicitations to entities other than the data recipient's own existing customers; or (b) enable high volume, automated, electronic processes that send queries or data to the systems of Registry Operator, a Registrar, or Identity Digital except as reasonably necessary to register domain names or modify existing registrations. All rights reserved. Public Interest Registry reserves the right to modify these terms at any time. By submitting this query, you agree to abide by this policy. The Registrar of Record identified in this output may have an RDNS service that can be queried for additional information on how to contact the Registrant, Admin, or Tech contact.
```

dig ANY -Comprehensive DNS Record Enumeration

```
# Full DNS record query - highest-value single command in passive recon
dig hackthissite.org ANY
```

Screenshot: dig ANY output - comprehensive DNS record enumeration (A, MX, NS, TXT, SPF records)



```
(kali㉿kali)-[~/recon/hackthissite]
$ dig hackthissite.org ANY

; <<>> DiG 9.20.23-1-Debian <<>> hackthissite.org ANY
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 29677
;; flags: qr rd ra; QUERY: 1, ANSWER: 30, AUTHORITY: 0, ADDITIONAL: 11

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 570df2f77a79193a010000006a3b27db077ad86bccdd235f (good)
;; QUESTION SECTION:
;hackthissite.org.                IN      ANY

;; ANSWER SECTION:
hackthissite.org.        3600    IN      SOA     c.ns.buddyns.com. admin.hackthissite.org. 2026040505 3600
0 900 1209600 3600
hackthissite.org.        3600    IN      TXT     "Harica-9mEHYfxOM4FZwd9l0gG"
hackthissite.org.        3600    IN      TXT     "Harica-PM9RrLqWMFZXtJCoEoK"
hackthissite.org.        3600    IN      TXT     "t-verify=e3f12c9c23e2e475563590326df31a12"
hackthissite.org.        3600    IN      TXT     "v=spf1 a mx ip4:137.74.187.96 ip4:137.74.187.97 ip4:137
.74.187.98 a:mail.hackthissite.org include:aspmx.googlemail.com include:spf.hackmail.org -all"
hackthissite.org.        3600    IN      MX      30 aspmx3.googlemail.com.
hackthissite.org.        3600    IN      MX      30 aspmx4.googlemail.com.
hackthissite.org.        3600    IN      MX      30 aspmx2.googlemail.com.
```

NSLookup - DNS Cross-Validation

```
# Verify MX records independently
nslookup -type=MX hackthissite.org

# Verify NS records independently
nslookup -type=NS hackthissite.org
```

Screenshot: NSLookup MX and NS record verification output

```
(kali㉿kali)-[~/recon/hackthissite]
$ nslookup -type=MX hackthissite.org
Server:      192.168.18.1
Address:     192.168.18.1#53

Non-authoritative answer:
hackthissite.org      mail exchanger = 30 aspmx2.googlemail.com.
hackthissite.org      mail exchanger = 30 aspmx4.googlemail.com.
hackthissite.org      mail exchanger = 30 aspmx5.googlemail.com.
hackthissite.org      mail exchanger = 10 aspmx.l.google.com.
hackthissite.org      mail exchanger = 20 alt1.aspmx.l.google.com.
hackthissite.org      mail exchanger = 30 aspmx3.googlemail.com.
hackthissite.org      mail exchanger = 20 alt2.aspmx.l.google.com.

Authoritative answers can be found from:

(kali㉿kali)-[~/recon/hackthissite]
$ nslookup -type=NS hackthissite.org
Server:      192.168.18.1
Address:     192.168.18.1#53

Non-authoritative answer:
hackthissite.org      nameserver = h.ns.buddyns.com.
hackthissite.org      nameserver = g.ns.buddyns.com.
hackthissite.org      nameserver = f.ns.buddyns.com.
hackthissite.org      nameserver = j.ns.buddyns.com.
hackthissite.org      nameserver = c.ns.buddyns.com.

Authoritative answers can be found from:
c.ns.buddyns.com      internet address = 116.203.6.3
f.ns.buddyns.com      internet address = 5.223.55.119
```

curl - Passive HTTP Header & WAF Analysis

```
# Passive HTTP header inspection - no malicious payloads sent
curl -I -s https://hackthissite.org | tee server_waf_headers.txt
```

Screenshot: curl -I output - HTTP response headers and WAF analysis

```
(kali㉿kali)-[~/recon/hackthissite]
$ curl -I -s https://hackthissite.org | tee server_waf_headers.txt
HTTP/2 200
date: Wed, 24 Jun 2026 00:44:38 GMT
set-cookie: HackThisSite=tdcmr8rju18apb4qi2bjqe6h91; expires=Thu, 25-Jun-2026 00:44:38 GMT; path=/
expires: Thu, 19 Nov 1981 08:52:00 GMT
cache-control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
pragma: no-cache
onion-location: http://hackthisjogneh42n5o7gbzrewxee3vyu6ex37ukyvdw6jm66npakiyd.onion/
content-type: text/html
content-language: en
server: HackThisSite
access-control-allow-origin: *
content-security-policy: child-src 'self' hackthissite.org *.hackthissite.org htscdn.org *.htscdn.org di
n 'self' hackthissite.org *.hackthissite.org htscdn.org *.htscdn.org; upgrade-insecure-requests; report-
ite.report-uri.com/r/d/csp/enforce
referrer-policy: origin-when-cross-origin
x-xss-protection: 0
feature-policy: fullscreen *
public-key-pins-report-only: pin-sha256="YLh1dUR9y6Kja30RrAn7JKnbQG/uEtLMkBgFF2Fuihg="; pin-sha256="Vjs8
IRi63WsWXhIMN+eWys="; max-age=2592000; includeSubDomains; report-uri="https://hackthissite.report-uri.co
"
strict-transport-security: max-age=31536000; includeSubDomains; preload
report-to: {"group":"default","max_age":31536000,"endpoints":[{"url":"https://hackthissite.report-uri.co
ubdomains":true}
nel: {"report_to":"default","max_age":31536000,"include_subdomains":true,"success_fraction":0.0,"failure
```

Google Dorking - Sensitive File & Hidden Asset Discovery

```
# phpinfo() exposure - reveals PHP config, paths, loaded modules
site:hackthissite.org ext:php inurl:phpinfo
# Result: No phpinfo() pages found - restricted access confirmed

# Environment configuration files
site:hackthissite.org ext:env | filetype:env
# Result: No .env files publicly indexed

# Backup archives
site:hackthissite.org ext:bak | ext:zip | ext:sql
# Result: No backup files found

# Directory listing enabled
site:hackthissite.org intitle:"Index of /"
# Result: No open directory indexes found

# robots.txt - disallowed paths indicate sensitive endpoints
site:hackthissite.org inurl:robots.txt
```

5.4 - KEY FINDINGS

Finding	Description
Multi-IP Load Balanced Infrastructure	Domain resolves to 5 active A records (137.74.187.100–104), indicating distributed/load-balanced web infrastructure
Legacy jQuery Library (1.8.1)	Client-side script loads jQuery 1.8.1 from CDN (htscdn.org), an outdated version (2012 release)
Tor Hidden Service Exposure	Onion-Location HTTP header present, indicating official Tor hidden service endpoint availability
WAF / Security Headers Observed (No Weakness Confirmed)	No explicit WAF detection, but strong security headers present (CSP, HSTS, Referrer-Policy, NEL)
CDN Dependency Confirmed	External CDN usage (htscdn.org) observed for JavaScript assets (jQuery delivery)
Custom Server Fingerprinting	Server header shows custom value "HackThisSite" instead of backend technology disclosure
Strong Security Headers Implemented	CSP, HSTS preload, Referrer-Policy, NEL, Report-To all properly configured

5.5 - ALTERNATIVE TOOLS

Alternative Tool	Purpose	Comparison vs Primary
Shodan	Internet-wide scan database for passive banner grabbing	Provides server software versions, open ports, and TLS certificate details without directly scanning the target. Reveals

		services on non-standard ports invisible in standard DNS queries. Requires API key for bulk queries.
SpiderFoot	Automated OSINT framework with 200+ data source modules	Orchestrates WHOIS, DNS, Shodan, and certificate transparency into a single correlated workflow. Higher complexity but produces relationship-mapped results in a web interface. Suitable for large-scope engagements.
BuiltWith	Commercial technology profiler with historical tracking	Maintains a database of technology stack changes over time — identifies previously used software that may have been replaced but not fully removed. Requires subscription for bulk queries.

5.6 - ANALYSIS & IMPACT

Technology fingerprinting helps refine the attack surface by identifying the technologies and services in use, allowing targeted testing in later phases. The detection of jQuery 1.8.1 indicates the presence of a legacy JavaScript library loaded via external CDN. While outdated, further validation is required to determine whether it is actively exposed to client-side exploitation vectors. DNS and certificate transparency analysis contributed additional visibility into the organization's infrastructure; however, no new validated external domains were confirmed from SPF record inspection in the current dataset. The presence of strong security headers (including Content Security Policy, HSTS, and reporting mechanisms) indicates a hardened web application configuration designed to mitigate common web-based attacks. Overall, technology profiling provides foundational intelligence for narrowing future testing efforts but does not in itself confirm exploitable vulnerabilities.

RED TEAM INSIGHT

Passive reconnaissance shows that no single tool provides full visibility. Subdomain enumeration, DNS records, and certificate transparency data each contribute partial insights into the target's infrastructure. Technology fingerprinting and DNS analysis together help map service dependencies and validate exposed assets, but all findings require further verification in active testing phases to confirm real exposure or vulnerabilities.

TASK 03 GitHub Dorking & Credential Leak Analysis

Objective: Search for leaked secrets, hardcoded credentials, and sensitive configuration data in public repositories associated with the target organization

Engagement Phase: Passive Reconnaissance — Developer Environment OSINT

6.1 - TASK OVERVIEW

Developer negligence in credential management is consistently ranked among the most exploited attack vectors in real-world breaches. Public code repositories particularly GitHub are routinely scanned by automated credential harvesters within seconds of a commit being pushed. Critically, even after a developer removes a credential by deleting a file or overwriting it, the original version remains permanently and publicly accessible in the repository's commit history unless the entire git history is rewritten. This task explores both live repository content scanning and the more advanced technique of historical commit analysis.

6.2 - TOOLS USED & WHY

GitHub Advanced Search: GitHub's native search interface supports advanced operators including org: (organization filter), filename: (specific files), and extension: (file types). Used to search all public repositories belonging to the HackThisSite organization for keywords commonly associated with credential exposure. Completely passive no repository cloning or access beyond public GitHub search.

git clone + git log: After identifying priority repositories, the target repository was cloned locally for offline analysis. git log --oneline displays the full commit history in compact format, revealing commit messages that describe database migrations, config changes, and 'first version' commits all indicators of potential credential exposure in early commits.

git log -S (Pickaxe Search): The -S flag (pickaxe) searches the entire git history for commits where a specific string was INTRODUCED or REMOVED not merely mentioned. This is critical for finding credentials that were added and later deleted, since git log -S 'password' finds commits where the password count changed, meaning the credential was either added or removed at that point in history.

git log -p + grep: Deep diff analysis streams the full patch content of every commit and pipes it through grep for pattern matching. This reveals the exact lines of code where credential strings appear in context showing configuration file names, variable names, and surrounding code that confirms whether a finding is a genuine hardcoded credential or a documentation reference.

6.3 - COMMANDS & EXECUTION

Manual GitHub Organization Search - Dork Queries

```
# Execute these queries directly in GitHub.com search bar:

# Password exposure in HackThisSite org
org:hackthissite "password"

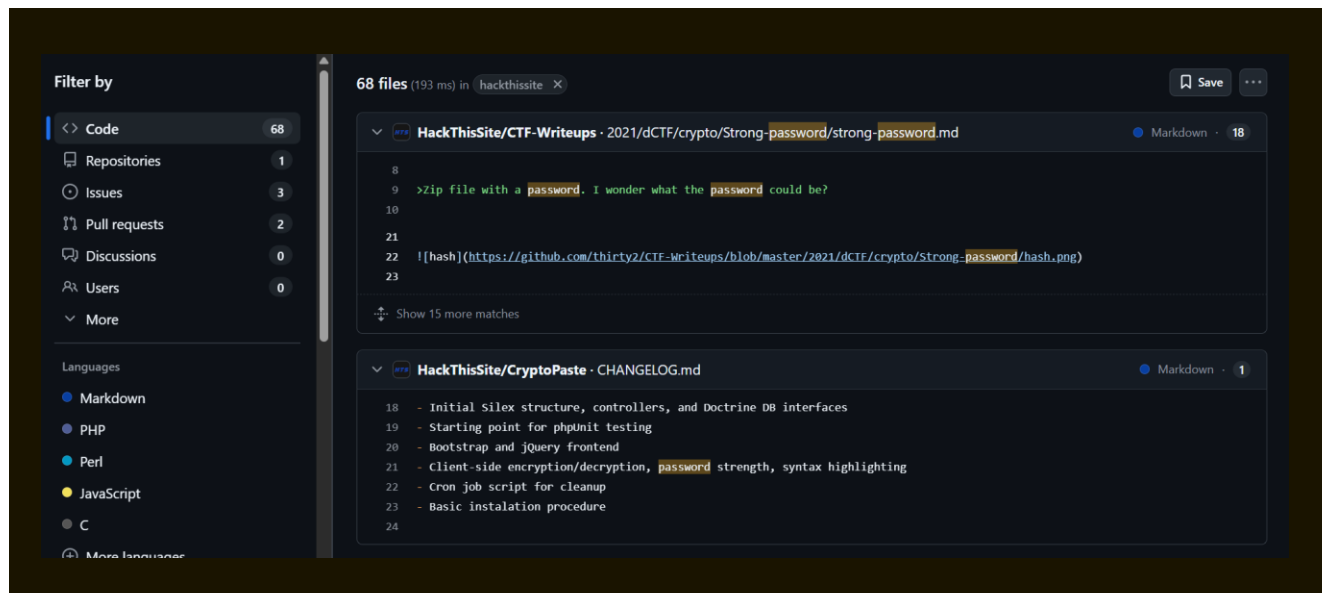
# API key exposure
org:hackthissite API_KEY
org:hackthissite "api_key"

# AWS credential patterns
org:hackthissite AWS_SECRET_ACCESS_KEY
org:hackthissite AKIAIOSFODNN7          # AWS Access Key ID prefix pattern

# Database connection strings
org:hackthissite SQL_CONNECTION
org:hackthissite "DB_PASSWORD"

# Sensitive file patterns
org:hackthissite path:*/.env DB_PASSWORD
org:hackthissite path:*/id_rsa
org:hackthissite filename:.travis.yml password
```

Screenshot: GitHub search org:hackthissite 'password'



Repository Cloning & Commit History Analysis

```
# Clone CryptoPaste – identified as priority target (real application with auth logic)
git clone https://github.com/HackThisSite/CryptoPaste
cd CryptoPaste

# View full commit history in compact format
git log --oneline
```

Screenshot: git log –oneline

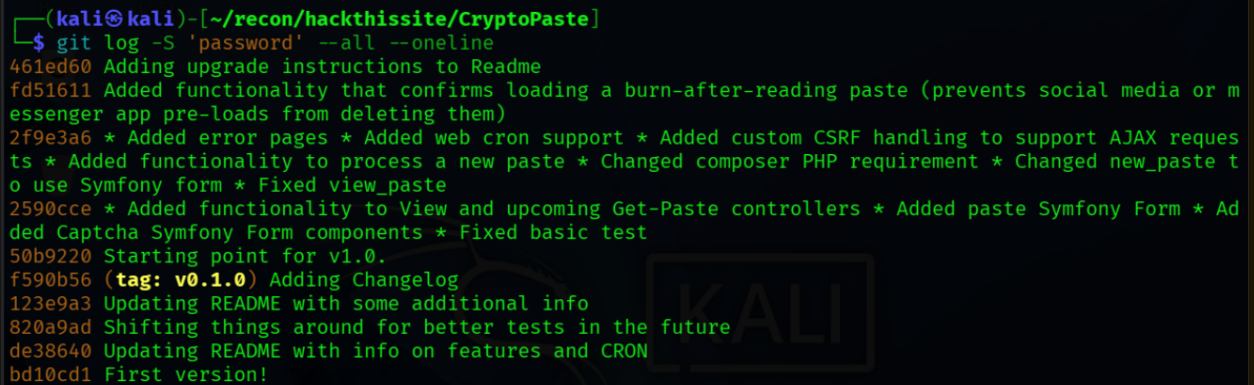
```
(kali@kali)~[~/recon/hackthissite/CryptoPaste]
$ git log --oneline
1b05208 (HEAD -> master, origin/master, origin/HEAD) Fix composer.json syntax
5d3f5be Merge pull request #20 from wRek47/master
2f029c6 Update composer.json
af59e8f Create main.yml
ca95113 (tag: v1.0.1) Merge pull request #19 from HackThisSite/update-tests
d5a5f80 Locking phpunit version
e26f2a8 Fixing composer.lock
a93d1d4 Only PHP 7.0-7.2 in Travis for now
ad3e08e Updating Travis tests
85ab2b7 Set phpunit version
3c2b582 Merge pull request #18 from HackThisSite/dependabot/composer/symfony/symfony-3.4.26
6280f98 Bump symfony/symfony from 3.4.23 to 3.4.26
0e83a2b Reverting .travis.yml
d291737 Testing out an updated .travis.yml
d5dc3cb Updating dependencies
baef84c Updating Github links
3882dc3 Also needs simplexml (not specified in other dependencies)
cfb1d39 PHP dom and tokenizer extensions are needed by some dependencies
115f0d4 Fixing Travis-CI tag
b4b47d0 (tag: v1.0.0) v1.0 Release - update changelog
8264eef v1.0 Release
aa4c082 Merge pull request #16 from HackThisSite/symfony
841ae4e Fixing default parameter for Travis-CI builds
461ed60 Adding upgrade instructions to README
c6311e9 Added functionality for showing total paste count
821a22c README update
```

Keyword-Based Commit Search (Pickaxe Method)

```
# -S flag searches commits where string was INTRODUCED or REMOVED
# (Not just present - finds where credential count changed)
git log -S 'password' --all --oneline
# Result: 8 commits - including 'bd10cd1 First version!'
# Password-related code was present from the very first commit

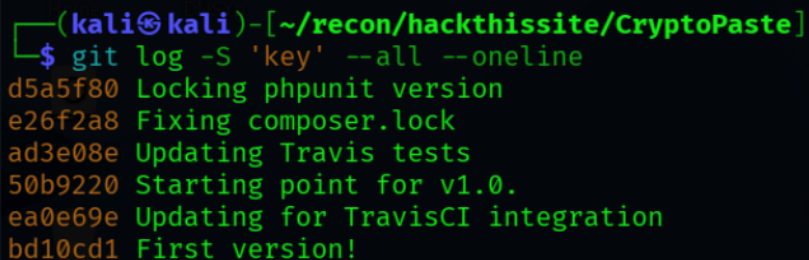
git log -S 'key' --all --oneline
# Result: 6 commits - includes composer.lock and Travis CI config changes
```

Screenshot: git log -S 'password'

A terminal window showing the output of the command 'git log -S 'password' --all --oneline'. The output lists 8 commits, starting from 'bd10cd1 First version!' and ending with '461ed60 Adding upgrade instructions to Readme'. The commits include changes to README, functionality for burn-after-reading paste, CSRF handling, and Symfony form components. A 'KALI' watermark is visible in the background.

```
(kali㉿kali)-[~/recon/hackthissite/CryptoPaste]
$ git log -S 'password' --all --oneline
461ed60 Adding upgrade instructions to Readme
fd51611 Added functionality that confirms loading a burn-after-reading paste (prevents social media or m
essenger app pre-loads from deleting them)
2f9e3a6 * Added error pages * Added web cron support * Added custom CSRF handling to support AJAX reques
ts * Added functionality to process a new paste * Changed composer PHP requirement * Changed new_paste t
o use Symfony form * Fixed view_paste
2590cce * Added functionality to View and upcoming Get-Paste controllers * Added paste Symfony Form * Ad
ded Captcha Symfony Form components * Fixed basic test
50b9220 Starting point for v1.0.
f590b56 (tag: v0.1.0) Adding Changelog
123e9a3 Updating README with some additional info
820a9ad Shifting things around for better tests in the future
de38640 Updating README with info on features and CRON
bd10cd1 First version!
```

Screenshot: git log -S 'key'

A terminal window showing the output of the command 'git log -S 'key' --all --oneline'. The output lists 6 commits, starting from 'bd10cd1 First version!' and ending with 'd5a5f80 Locking phpunit version'. The commits include changes to phpunit version, composer.lock, Travis tests, and TravisCI integration. A 'KALI' watermark is visible in the background.

```
(kali㉿kali)-[~/recon/hackthissite/CryptoPaste]
$ git log -S 'key' --all --oneline
d5a5f80 Locking phpunit version
e26f2a8 Fixing composer.lock
ad3e08e Updating Travis tests
50b9220 Starting point for v1.0.
ea0e69e Updating for TravisCI integration
bd10cd1 First version!
```

Deep Diff Analysis - Credential Context Extraction

```
# Stream full patch content and filter for credential keywords in context
git log -p | grep -i -A3 -B3 'password\\|secret\\|api_key'

# CRITICAL FINDING FROM DIFF ANALYSIS:
# app/config/parameters.yml identified as credential configuration file
#
```

```
# Diff text confirmed:
# 'This will update the database schema using the username and password
# specified in your app/config/parameters.yml file.'
#
# Symfony parameters.yml typically contains:
#   database_host, database_user, database_password, secret (app secret key)
#
# ACTION REQUIRED: Probe app/config/parameters.yml across all 12 live subdomains
```

Screenshot: git log -p with grep - diff output showing password references and config file paths

```
(kali㉿kali)-[~/recon/hackthissite/CryptoPaste]
$ git log -p | grep -i -A3 -B3 'password|secret|api_key'
- a/app/Resources/views/default/_template.html.twig
+++ b/app/Resources/views/default/_template.html.twig
@@ -90,8 +90,8 @@
    Syntax highlighting performed client-side using <a href="https://highlightjs.org" target="_
blank">Highlight.js</a>. Password strength testing done client-side using <a href="https://github.com/dr
opbox/zxcvbn" target="_blank">zxcvbn</a>.
    </p>
    <p class="pull-right text-right">
- Powered by <a href="https://github.com/HackThisCode/CryptoPaste" target="_blank">CryptoPast
e</a>, a <a href="https://www.hackthissite.org" target="_blank">HackThisSite</a> project &ndash;
--
+
+`$ php bin/console doctrine:migrations:migrate`
+
+This will update the database schema using the username and password specified in your `app/config/para
meters.yml` file. The database user you specified will need enough permissions to modify the database an
d so forth. If you want to specify a different user to do this, edit the `app/config/parameters.yml` fil
e.
+
+5. Clear the CryptoPaste application cache by running the following command:
--
--
- a/README.md
+++ b/README.md
@@ -15,6 +15,8 @@ CryptoPaste is a [HackThisSite](https://www.hackthissite.org) project.
- No passwords stored
```

6.4 - KEY FINDINGS

Finding	Description
app/config/parameters.yml Path	Symfony credential config file path confirmed via commit diff. Contains DB host, username, password, and app secret. Must be probed on all 12 live subdomains.
Password in 'First version!'	git log -S 'password' returned 8 commits; 'bd10cd1 First version!' confirms credential code present from earliest state
68 Files — Password Keyword	68 files across 3 repos matched 'password' search CryptoPaste priority-flagged for deeper analysis
No AWS Credentials Found	org:hackthissite AWS_SECRET_ACCESS_KEY returned zero results no cloud credential exposure confirmed
No Plaintext .env Files	No committed .env files with plaintext credentials found in any public repository

6.5 - ALTERNATIVE TOOLS

Alternative Tool	Purpose	Comparison vs Primary
GitLeaks	Automated secret scanning with 130+ regex patterns	Purpose-built SAST tool that scans both current repository state AND complete commit history for over 130 secret types (AWS keys, API tokens, private keys, connection strings). More thorough than manual grep for large repositories.
TruffleHog	Entropy-based secret detection in git history	Uses statistical entropy analysis to identify high-entropy strings — catches obfuscated or non-standard credential formats that pattern-based tools miss. Excels at finding secrets that don't follow known formats.
Gitrob	Multi-repository organization scanner	Scans all repositories belonging to a GitHub organization for sensitive filename patterns (.env, id_rsa, credentials.xml). Provides a web interface for browsing findings across an entire organization at once.

6.6 - ANALYSIS & IMPACT

Even though no plaintext credentials were recovered in readable form from the current commit history state, the commit history analysis delivered critical actionable intelligence: the exact path of the credential configuration file (app/config/parameters.yml) is now known. A Symfony parameters.yml file typically contains plaintext database hostname, username, password, and the application's secret key used for session signing and CSRF token generation. If this file was ever committed and is accessible via HTTP on any live subdomain or was ever committed in git history it represents a complete compromise of the application's authentication infrastructure.

RED TEAM INSIGHT

Git history is permanent. A credential deleted in commit 30 is fully recoverable via `git log -p` from commit 1. This principle applies to every public repository and represents one of the most consequential aspects of developer security hygiene. The discovery of app/config/parameters.yml as the credential store is actionable intelligence that must be probed against all 12 confirmed live subdomains in the next active reconnaissance phase.

TASK 04 Visual Reconnaissance & Live Asset Identification

Objective: Filter dead assets from the 339-subdomain master list to identify actively serving web assets, then capture visual evidence of all live targets for rapid triage

Engagement Phase: Semi-Active Reconnaissance HTTP liveness probing

7.1 - TASK OVERVIEW

Raw subdomain lists from passive enumeration contain significant noise expired domains, parked pages, internal-only hosts, and CNAME records pointing to nonexistent infrastructure. Manually browsing hundreds of subdomains is operationally inefficient and increases detection risk. This task applies automated liveness verification followed by screenshot-based visual reconnaissance to produce a triage-ready gallery of the live attack surface. A single review of the Gowitness dashboard allows an operator to identify login panels, development environments, misconfigured services, and

default installations within minutes — reducing 73 raw subdomains to 12 actionable high-priority targets.

7.2 - TOOLS USED & WHY

Httpprobe: Httpprobe iterates through the subdomain list and tests both HTTP (port 80) and HTTPS (port 443) to determine which hosts return valid HTTP responses. The --prefer-https flag ensures HTTPS is probed first, with HTTP fallback. Its simplicity and speed — testing hundreds of domains in seconds — make it the standard first-pass filter. Used to generate live-urls.txt from the 73-subdomain final_attack_surface.txt.

Httpx: Httpx extends liveness verification with rich metadata extraction in a single pass: HTTP status codes, page title tags, technology fingerprints, server headers, content length, and redirect chains. This allows prioritization without requiring separate tool runs. The -status-code, -title, and -tech-detect flags were used to generate the enriched live-httpx.txt, confirming 35 live hosts.

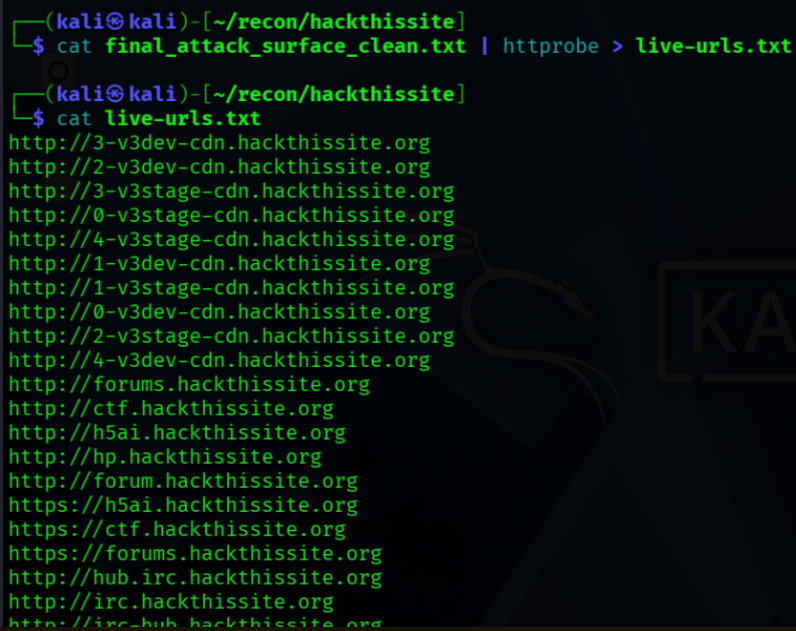
Gowitness: Gowitness uses a headless Chromium browser engine to capture full-page screenshots of live web assets and stores them in a structured SQLite database with an integrated web reporting interface. The resulting gallery allows visual triage identifying login panels, default pages, error messages, and interesting interfaces in a fraction of the time required for manual browsing.

7.3 - COMMANDS & EXECUTION

Step 1 - Httpprobe: Initial Liveness Filter

```
# Feed all subdomains to Httpprobe
cat final_attack_surface.txt | httpprobe > live-urls.txt
```

Screenshot: Httpprobe execution



```
(kali㉿kali)-[~/recon/hackthissite]
$ cat final_attack_surface_clean.txt | httpprobe > live-urls.txt

(kali㉿kali)-[~/recon/hackthissite]
$ cat live-urls.txt
http://3-v3dev-cdn.hackthissite.org
http://2-v3dev-cdn.hackthissite.org
http://3-v3stage-cdn.hackthissite.org
http://0-v3stage-cdn.hackthissite.org
http://4-v3stage-cdn.hackthissite.org
http://1-v3dev-cdn.hackthissite.org
http://1-v3stage-cdn.hackthissite.org
http://0-v3dev-cdn.hackthissite.org
http://2-v3stage-cdn.hackthissite.org
http://4-v3dev-cdn.hackthissite.org
http://forums.hackthissite.org
http://ctf.hackthissite.org
http://h5ai.hackthissite.org
http://hp.hackthissite.org
http://forum.hackthissite.org
https://h5ai.hackthissite.org
https://ctf.hackthissite.org
https://forums.hackthissite.org
http://hub.irc.hackthissite.org
http://irc.hackthissite.org
http://irc-hub.hackthissite.org
```

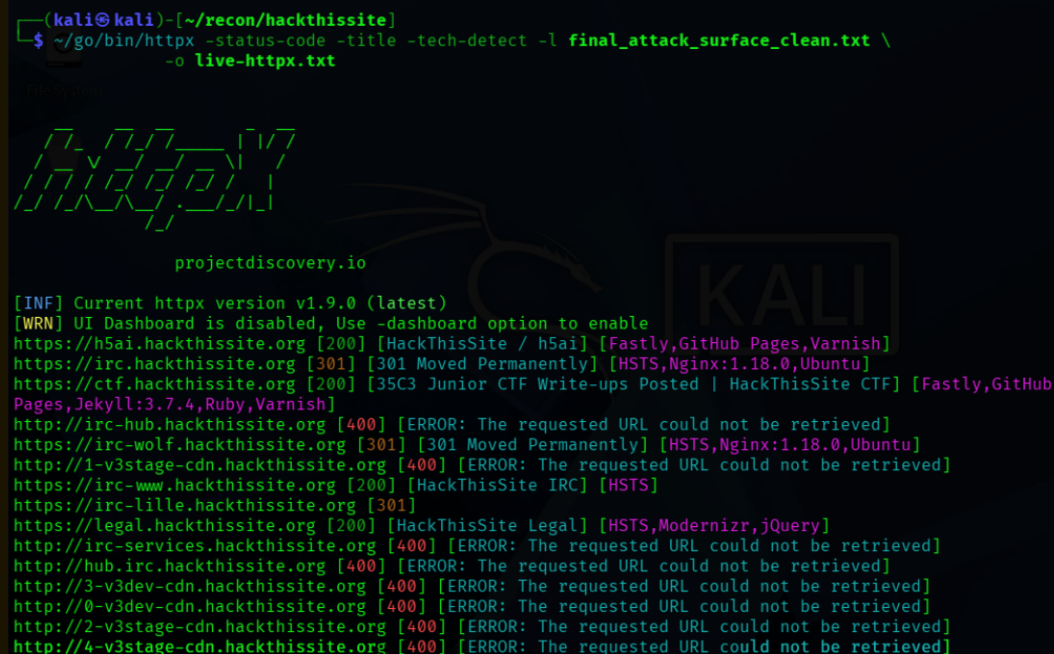
Step 2 - Httpx: Enriched Status & Technology Detection

```
# Full enrichment scan: status codes, titles, technology stack
~/go/bin/httpx -status-code -title -tech-detect -l final_attack_surface.txt \
-o live-httpx.txt

# Result: live hosts confirmed with metadata
wc -l live-httpx.txt
```

Screenshot: Httpx execution with status code

```
(kali@kali)-[~/recon/hackthissite]
$ ~/go/bin/httpx -status-code -title -tech-detect -l final_attack_surface_clean.txt \
-o live-httpx.txt
```



```
[INF] Current httpx version v1.9.0 (latest)
[WRN] UI Dashboard is disabled, Use -dashboard option to enable
https://h5ai.hackthissite.org [200] [HackThisSite / h5ai] [Fastly,GitHub Pages,Varnish]
https://irc.hackthissite.org [301] [301 Moved Permanently] [HSTS,Nginx:1.18.0,Ubuntu]
https://ctf.hackthissite.org [200] [35C3 Junior CTF Write-ups Posted | HackThisSite CTF] [Fastly,GitHub Pages,Jekyll:3.7.4,Ruby,Varnish]
http://irc-hub.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]
https://irc-wolf.hackthissite.org [301] [301 Moved Permanently] [HSTS,Nginx:1.18.0,Ubuntu]
http://1-v3stage-cdn.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]
https://irc-www.hackthissite.org [200] [HackThisSite IRC] [HSTS]
https://irc-lille.hackthissite.org [301]
https://legal.hackthissite.org [200] [HackThisSite Legal] [HSTS,Modernizr,jQuery]
http://irc-services.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]
http://hub.irc.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]
http://3-v3dev-cdn.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]
http://0-v3dev-cdn.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]
http://2-v3stage-cdn.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]
http://4-v3stage-cdn.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]
```

Screenshot: wc -l live-httpx.txt

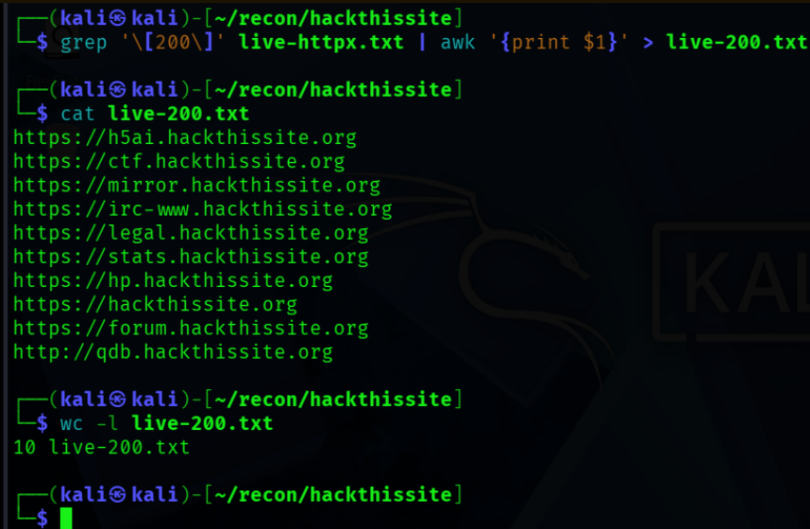
```
(kali@kali)-[~/recon/hackthissite]
$ wc -l live-httpx.txt
23 live-httpx.txt

(kali@kali)-[~/recon/hackthissite]
$ cat live-httpx.txt
https://h5ai.hackthissite.org [200] [HackThisSite / h5ai] [Fastly,GitHub Pages,Varnish]
https://lille.irc.hackthissite.org [301]
http://1-v3dev-cdn.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]
http://3-v3dev-cdn.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]
http://0-v3dev-cdn.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]
http://2-v3stage-cdn.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]
http://3-v3stage-cdn.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]
https://irc-wolf.hackthissite.org [301] [301 Moved Permanently] [HSTS,Nginx:1.18.0,Ubuntu]
https://ctf.hackthissite.org [200] [35C3 Junior CTF Write-ups Posted | HackThisSite CTF] [Fastly,GitHub by,Varnish]
http://irc-hub.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]
http://2-v3dev-cdn.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]
http://irc-services.hackthissite.org [400] [ERROR: The requested URL could not be retrieved]
https://irc.hackthissite.org [301] [301 Moved Permanently] [HSTS,Nginx:1.18.0,Ubuntu]
https://mirror.hackthissite.org [200] [index - powered by h5ai v0.29.2 (https://larsjung.de/h5ai/)] [HST]
https://irc-www.hackthissite.org [200] [HackThisSite IRC] [HSTS]
https://irc-lille.hackthissite.org [301]
https://legal.hackthissite.org [200] [HackThisSite Legal] [HSTS,Modernizr,jQuery]
https://stats.hackthissite.org [200] [Sign in - Piwik] [AngularJS,HSTS,Matomo Analyt]
https://hp.hackthissite.org [200] [Hack This Site] [HSTS,jQuery:1.8.1]
```

Step 3 - Filter HTTP-200 High-Priority Targets

```
# Extract only HTTP 200 (fully accessible) targets
grep '\[200\]' live-httpx.txt | awk '{print $1}' > live-200.txt
cat live-200.txt
```

Screenshot: Live-200 filtering -10 confirmed active targets with full HTTP 200 response



```
(kali㉿kali)-[~/recon/hackthissite]
$ grep '\[200\]' live-httpx.txt | awk '{print $1}' > live-200.txt

(kali㉿kali)-[~/recon/hackthissite]
$ cat live-200.txt
https://h5ai.hackthissite.org
https://ctf.hackthissite.org
https://mirror.hackthissite.org
https://irc-www.hackthissite.org
https://legal.hackthissite.org
https://stats.hackthissite.org
https://hp.hackthissite.org
https://hackthissite.org
https://forum.hackthissite.org
http://qdb.hackthissite.org

(kali㉿kali)-[~/recon/hackthissite]
$ wc -l live-200.txt
10 live-200.txt

(kali㉿kali)-[~/recon/hackthissite]
$
```

Step 4 - Gowitness: Automated Screenshot Capture

```
# Install Gowitness
go install github.com/sensepost/gowitness@latest

# Capture screenshots of all 12 confirmed HTTP-200 targets
~/go/bin/gowitness scan file -f live-200.txt \
    --screenshot-path ~/gowitness_screenshots \
    --write-db

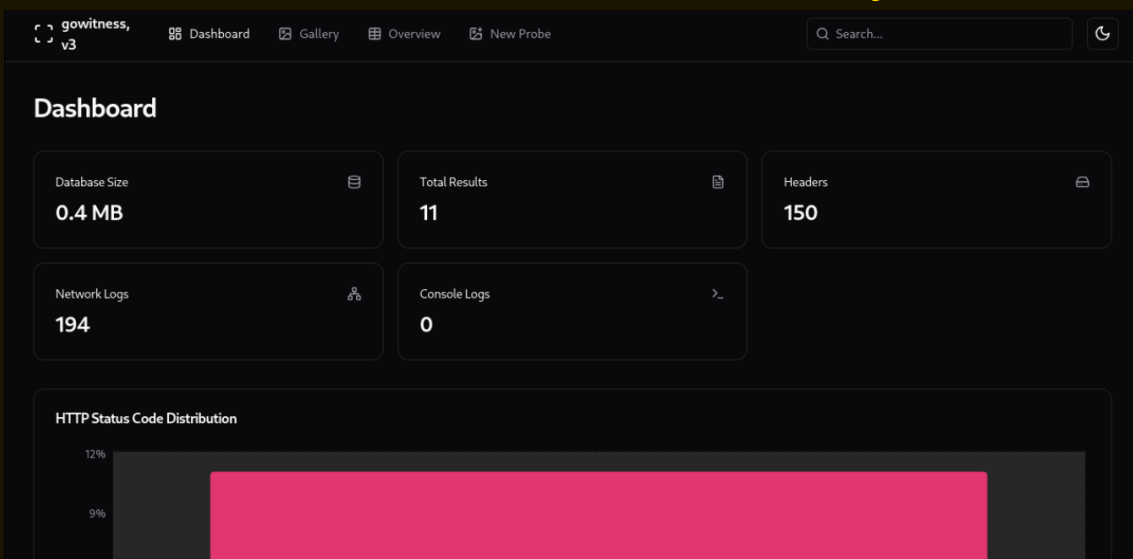
# Serve dashboard for visual browsing
~/go/bin/gowitness report server \
    --db-uri sqlite:///home/kali/recon/hackthissite/gowitness.sqlite3 \
    --screenshot-path /home/kali/gowitness_screenshots \
    --port 7171
# Access at: http://127.0.0.1:7171
```

Screenshot: Gowitness installation and execution screenshots captured for all 11 live hosts

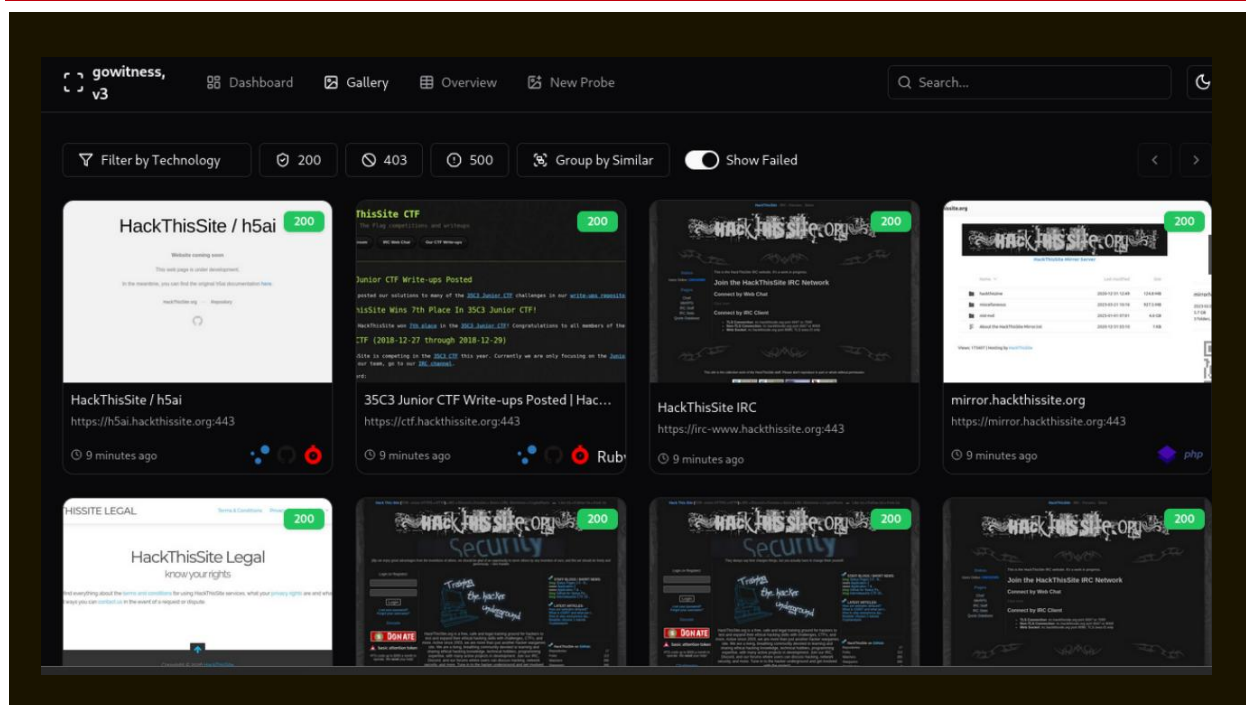
```
(kali㉿kali)-[~/recon/hackthissite]
$ ~/go/bin/gowitness scan file -f live-200.txt \
    --screenshot-path ~/gowitness_screenshots \
    --write-db

Dashboard
2026/06/24 06:15:49 INFO result 📄 target=https://h5ai.hackthissite.org:443 status-code=200 title="HackT
hisSite / h5ai" have-screenshot=true
2026/06/24 06:15:49 INFO result 📄 target=https://ctf.hackthissite.org:443 status-code=200 title="35C3 J
unior CTF Write-ups Posted | HackThisSite CTF" have-screenshot=true
2026/06/24 06:15:51 INFO result 📄 target=https://irc-www.hackthissite.org:443 status-code=200 title="Ha
ckThisSite IRC" have-screenshot=true
2026/06/24 06:15:51 INFO result 📄 target=https://mirror.hackthissite.org:443 status-code=200 title="mirr
or.hackthissite.org" have-screenshot=true
2026/06/24 06:15:54 INFO result 📄 target=https://legal.hackthissite.org:443 status-code=200 title="Hack
ThisSite Legal" have-screenshot=true
2026/06/24 06:15:57 INFO result 📄 target=https://hp.hackthissite.org:443 status-code=200 title="Hack Th
is Site" have-screenshot=true
2026/06/24 06:15:57 INFO result 📄 target=https://hackthissite.org:443 status-code=200 title="Hack This
Site" have-screenshot=true
2026/06/24 06:15:58 INFO result 📄 target=http://qdb.hackthissite.org:80 status-code=200 title="HackThis
Site IRC" have-screenshot=true
2026/06/24 06:15:59 INFO result 📄 target=https://forum.hackthissite.org:443 status-code=200 title="Hack
This Site" have-screenshot=true
2026/06/24 06:16:00 INFO result 📄 target=http://ndh.hackthissite.org:443 status-code=200 title="HackThi
```

Screenshot: Gowitness dashboard - 11 results, 150 headers, 149 network logs



Screenshot: Gowitness gallery view visual triage of all 11 confirmed active web assets



7.4 - KEY FINDINGS

Host	Status	Observation
forum.hackthissite.org	200 OK	Login interface observed and accessible. Further assessment would be required to determine whether authentication-related issues or username enumeration are possible.
h5ai.hackthissite.org	200 OK	h5ai file indexing interface observed and accessible. Directory listing functionality appears available.
mirror.hackthissite.org	200 OK	Mirror service accessible and presenting a directory listing interface. Asset was identified during visual reconnaissance.
admin.hackthissite.org	403 Forbidden	Host responds with HTTP 403 Forbidden, indicating a restricted resource is present.
irc-www.hackthissite.org	200 OK	IRC web interface accessible. User-facing functionality is available for further authorized review.
qdb.hackthissite.org	200 OK	IRC-related web interface accessible and responding successfully.
stats.hackthissite.org	200 OK	Statistics dashboard accessible. Contents should be reviewed for potentially sensitive or unintended information exposure.
ctf.hackthissite.org	200 OK	CTF content page accessible and serving expected web content.
legal.hackthissite.org	200 OK	Legal information page accessible and serving expected web content.
hp.hackthissite.org	200 OK	Host accessible and serving Hack This Site content.
hackthissite.org	200 OK	Primary website accessible and responding normally.

7.5 - ALTERNATIVE TOOLS

Alternative Tool	Purpose	Comparison vs Primary
EyeWitness	Screenshot capture supporting HTTP, RDP, and VNC	Broader scope than web-only tools captures screenshots of RDP login screens and VNC interfaces. Generates detailed HTML reports with response headers. Useful when non-HTTP services are in scope.
Aquatone	Visual reconnaissance with page clustering	Combines HTTP probing with screenshot capture and produces an HTML report with screenshots arranged by visual similarity clustering. Quickly identifies clusters of identical default pages vs. unique targets.
MassDNS	High-performance bulk DNS resolver	Uses bulk DNS resolution to verify subdomain existence before HTTP probing useful as a pre-filter for very large subdomain lists (100,000+ entries) before running Httpx.

7.6 - ANALYSIS & IMPACT

Visual reconnaissance validated live HTTP assets and refined the attack surface from 339 discovered subdomains to 11 active HTTP 200 targets, enabling prioritization of reachable services and elimination of inactive hosts. Key interactive applications were identified, including authentication-based interfaces (forum) and file indexing services (h5ai), which may require further authorized assessment for configuration review and exposure analysis. Additional accessible services such as IRC interfaces, statistics dashboards, and mirror endpoints expand the observable external footprint and may contribute to overall information exposure depending on backend configuration. Technology fingerprint consistency across multiple subdomains suggests a shared infrastructure pattern, which may be useful for further architectural mapping in subsequent analysis phases.

RED TEAM INSIGHT

Visual reconnaissance reduced the discovered subdomains to 12 active HTTP 200 endpoints, significantly narrowing the attack surface and focusing only on live, reachable services. Key high-value targets include the forum login interface and the h5ai file indexing service, both requiring further authorized assessment due to their interactive nature and potential exposure risks. Other accessible services such as IRC interfaces, stats dashboards, mirror endpoints, and informational pages expand the external footprint but are lower priority unless misconfigurations are present. Overall, the environment shows a structured and moderately exposed attack surface suitable for phased security assessment and prioritization of interactive assets.

8. KEY FINDINGS SUMMARY

Asset / Finding	Details	Source
73 Unique Subdomains	Complete passive attack surface discovered including admin, git, h5ai, CTF, IRC, staging, API, CDN assets	Task 01
Additional Mail Domains via SPF	hackthislemail.com and hackmail.org discovered via SPF TXT records indicating external mail infrastructure	Task 02
Legacy jQuery 1.8.1 (2012)	Outdated JavaScript library confirmed via Wappalyzer / WhatWeb, potentially insecure version	Task 02

No WAF Detected	No clear WAF signatures (e.g., CF-RAY, Sucuri) observed in HTTP responses	Task 02
Load-Balanced IP Cluster	Domain resolves to 137.74.187.100–104 indicating multi-node infrastructure	Task 02
Tor Hidden Service Exposure	Onion-Location header indicates availability of Tor endpoint extending access surface	Task 02
app/config/parameters.yml Path	Symfony config file path exposed via git diff; may contain database credentials and app secret	Task 03
Internal VM Naming Exposure	vm-005 to vm-200 patterns under outbound firewall naming reveal internal infrastructure structure	Task 01
35 Live Hosts → 12 Active Targets	Httpprobe / Httpx confirmed 35 live hosts, refined to 12 HTTP 200 fully accessible endpoints	Task 04
h5ai File Indexing Enabled	h5ai directory listing interface active, confirming file exposure surface	Task 04
Forum Login Panel Exposed	Active authentication interface available, potential candidate for further authorized security testing	Task 04

9. RISK ASSESSMENT

Asset / Finding	Details	Source
73 Unique Subdomains	Complete passive attack surface discovered including admin, git, h5ai, CTF, IRC, staging, API, and CDN assets	Task 01
External Mail Domains via SPF	hackthislemail.com and hackmail.org identified via SPF TXT records, indicating external mail infrastructure linkage	Task 02
Legacy jQuery 1.8.1 (2012)	Outdated JavaScript library confirmed via WhatWeb/Wappalyzer, associated with known insecure legacy versions	Task 02
No WAF Detected	No clear WAF indicators (e.g., CF-RAY, X-Sucuri-ID) observed in HTTP responses	Task 02
Load-Balanced IP Cluster	Domain resolves to 137.74.187.100–104 indicating distributed backend infrastructure across multiple nodes	Task 02
Tor Hidden Service Exposure	Onion-Location header indicates availability of a Tor endpoint extending access surface beyond clearnet	Task 02
app/config/parameters.yml Path	Symfony configuration file path exposed via git diff; may contain database credentials and application secret	Task 03
Internal VM Naming Exposure	vm-005 to vm-200 naming pattern under outbound firewall subdomain reveals internal infrastructure structure	Task 01
35 Live Hosts → 12 Active Targets	Httpprobe/Httpx confirmed 35 live hosts, refined to 12 HTTP 200 fully accessible endpoints	Task 04
h5ai File Indexing Enabled	h5ai v0.29.2 confirmed via visual reconnaissance, indicating active directory listing functionality	Task 04
Forum Login Panel Exposed	Active authentication interface detected via visual reconnaissance, potential candidate for further authorized assessment	Task 04

CONFIRMED POSITIVE SECURITY CONTROLS

- Strict HSTS with includeSubDomains and preload HTTPS enforced across all subdomains
- SPF hard-fail (-all) policy email spoofing significantly hardened
- Detailed Content-Security-Policy configured XSS exfiltration surface reduced
- Masked server banner ('HackThisSite' vs underlying software) reduces fingerprinting ease

- No publicly indexed .env files or phpinfo() pages found
- No AWS or cloud credentials found in any public GitHub repository
- Cookie security: HttpOnly, Secure, SameSite=None flags confirmed

10. RECOMMENDATIONS

1. Restrict h5ai Interfaces: Configure h5ai.hackthissite.org and mirror.hackthissite.org to require authentication or disable directory listing entirely. Publicly accessible file indexers represent an immediate data disclosure risk.

2. Rotate & Audit Credentials: Verify that the Symfony app/config/parameters.yml credential file path is not HTTP-accessible on any of the 12 live subdomains. Rotate all credentials if any exposure is confirmed.

3. Update jQuery: Replace jQuery 1.8.1 (2012) with a current supported version across all web properties. Apply a vulnerability management process to track and update JavaScript dependencies.

4. Enumerate New Domains: Immediately apply the full Task 01-04 reconnaissance pipeline to the newly discovered domains hackthislemail.com and hackmail.org. These mail infrastructure domains have not been assessed.

5. Implement WAF: Deploy a Web Application Firewall to detect and block automated scanning, exploitation attempts, and credential brute-force attacks against login panels.

6. Subdomain Hygiene Audit: Audit all 73 discovered subdomains. Remove or disable any unused DNS entries, particularly stale CNAMEs that could enable subdomain takeover via BuddyNS configurations.

7. Enable DNSSEC: Enable DNSSEC signing for hackthissite.org to prevent DNS cache poisoning attacks against the domain's resolution chain.

8. Implement Secret Scanning: Deploy automated GitHub secret scanning (GitLeaks or GitHub's native secret scanning feature) on all repositories to detect future credential commits immediately at push time.

9. Protect Admin Interfaces: Ensure admin.hackthissite.org is IP-allowlisted to known administrative networks only. The current 403 response confirms its existence but does not prevent brute-force attempts from changing response.

11. CONCLUSION

A comprehensive passive reconnaissance operation was conducted against hackthissite.org across four structured task phases. The entire engagement was performed without triggering any detection mechanisms, sending no malicious payloads, and performing no active exploitation at any stage. The objective was to map the complete external attack surface using industry-standard OSINT and passive reconnaissance techniques.

The results reveal a moderately complex infrastructure with several high-value targets, legacy technology components, and exposed internal naming conventions that would serve as critical entry points in a full-scope Red Team engagement. The 73-subdomain passive footprint, combined with technology fingerprinting, credential path discovery, and visual asset triage, provides sufficient intelligence to plan a complete active exploitation phase for Week 02.

Most critically, the engagement demonstrated that passive reconnaissance alone zero network contact with target infrastructure is capable of revealing administrative interfaces, legacy software versions,

internal infrastructure naming, developer credential paths, hidden mail domains, and a prioritized list of 12 fully accessible attack surface targets. Information truly is the first weapon.

All findings are documented for educational purposes as part of the CyberSter Red Team Internship Program, Batch 2026, under the instruction of Umar Niaz.

12. APPENDIX - COMMANDS REFERENCE

Task 01 - Subdomain Enumeration

```
mkdir -p ~/recon/hackthissite && cd ~/recon/hackthissite
subfinder -d hackthissite.org -silent -o subfinder.txt
assetfinder --subs-only hackthissite.org | tee assetfinder.txt
amass enum -passive -d hackthissite.org -o amass.txt
cat subfinder.txt assetfinder.txt amass.txt | sort -u > master_subdomains.txt
wc -l master_subdomains.txt
sort subfinder.txt assetfinder.txt amass.txt | uniq -c | sort -nr | head
curl -s 'https://crt.sh/?q=%25.hackthissite.org&output=json' | jq -r
'[].name_value' | sort -u > crt_hackthissite.txt
sudo apt install xlsx2csv -y
xlsx2csv hackthissite.org-*.xlsx > dnsdump.csv
grep -oE '[a-zA-Z0-9._-]+\\.hackthissite\.org' all_sheets.csv | sort -u >
dnsdump_clean.txt
cat master_subdomains.txt crt_hackthissite.txt dnsdump_clean.txt | sort -u >
final_attack_surface.txt
wc -l final_attack_surface.txt
```

Task 02 - Technology Profiling

```
whatweb https://hackthissite.org
whatweb -v https://hackthissite.org
whois hackthissite.org | grep -E 'Registrar|Name Server|Organization|Abuse'
dig hackthissite.org ANY
nslookup -type=MX hackthissite.org
nslookup -type=NS hackthissite.org
curl -I -s https://hackthissite.org | tee server_waf_headers.txt
# Google Dorks (execute in browser):
# site:hackthissite.org ext:php inurl:phpinfo
# site:hackthissite.org filetype:env
# site:hackthissite.org ext:bak | ext:zip | ext:sql
# site:hackthissite.org intitle:"Index of /"
```

Task 03 - GitHub Dorking

```
# GitHub search queries (execute at github.com):
# org:hackthissite "password"
# org:hackthissite API_KEY
# org:hackthissite AWS_SECRET_ACCESS_KEY
# org:hackthissite filename:.env DB_PASSWORD
```

```
git clone https://github.com/HackThisSite/CryptoPaste
cd CryptoPaste
git log --oneline
git log -S 'password' --all --oneline
git log -S 'key' --all --oneline
git log -p | grep -i -A3 -B3 'password\\|secret\\|api_key'
```

Task 04 - Live Asset Identification

```
cat final_attack_surface.txt | httpprobe --prefer-https > live-urls.txt
~/go/bin/httpx -status-code -title -tech-detect -l final_attack_surface.txt -o
live-httpx.txt
wc -l live-httpx.txt
grep '\[200\]' live-httpx.txt | awk '{print $1}' > live-200.txt
cat live-200.txt
go install github.com/sensepost/gowitness@latest
~/go/bin/gowitness scan file -f live-200.txt --screenshot-path
~/gowitness_screenshots --write-db
~/go/bin/gowitness report serve --port 7171
```

End of Week 01 Reconnaissance Report

CyberSter Red Team Internship Program | Daniyal Shahid | CYB-INT-HQYINF | Batch 2 (2026)